



INSTRUMENTO DE EVALUACIÓN

Servicio Nacional de Aprendizaje – SENA

Programa de Formación: Tecnólogo en Análisis y Desarrollo de Software

Código: 228118 – Versión 1

Ficha: 3151895

Competencia: Desarrollar la solución de software de acuerdo con el diseño y metodologías de desarrollo.

Resultado de Aprendizaje a evaluar: Codificar el software de acuerdo con el diseño establecido.

Instrumento: Ejercicio práctico de codificación

Duración total: 2 horas sección práctica,

Entorno para la parte práctica: Visual Studio Code, Python 3.x, SQLite 3, HTML5, CSS3

INSTRUCCIONES PARA EL APRENDIZ

- Lea cuidadosamente cada pregunta antes de responder.
- La sección teórica no permite el uso de computador, celular ni apuntes.
- La sección práctica se realiza en el computador asignado, guardando el proyecto en la carpeta indicada por el instructor.
- Administre su tiempo: se recomienda no more de 10 minutos por bloque en la parte teórica y seguir el orden sugerido en la parte práctica.
- Escriba con letra clara en las preguntas abiertas y de autocompletar.

SECCIÓN 2 — EVALUACIÓN PRÁCTICA

(Duración: 2 horas — Codificación individual en VS Code — Python + SQLite + HTML + CSS)

Nombre del aprendiz: Juan David Trujillo Saenz **Fecha:** 10/08/26
CC.TI: 1081405020 **Ficha:** 3151895

SITUACIÓN PROBLEMA

Contexto: *Centro Veterinario PatitasSanas* requiere digitalizar el registro de sus pacientes (mascotas atendidas). Actualmente los datos se llevan en una hoja de cálculo compartida por varios auxiliares, lo que ha generado registros duplicados y pérdida de información. El diseño de la solución (modelo de datos y mockup de pantalla) ya fue aprobado previamente por el analista del proyecto; a usted, como desarrollador, le corresponde **codificar** el módulo de "Gestión de Pacientes" siguiendo ese diseño.



SERVICIO NACIONAL DE APRENDIZAJE SENA
Centro de Desarrollo Agroempresarial y Turístico del HUILA
Instrumento de evaluación conocimientos de proceso codificación
Tecnólogo en Análisis y Desarrollo de Software



Diseño ya aprobado, que usted debe codificar:

Tabla `pacientes`, con los siguientes atributos:

Atributo	Tipo	Restricción
id	Entero	Clave primaria, autoincremental
nombre_mascota	Texto	Obligatorio
especie	Texto	Obligatorio
edad	Entero	Debe ser mayor o igual a 0
nombre_propietario	Texto	Obligatorio
telefono_propietario	Texto	Obligatorio

Mockup de la pantalla principal (diseño ya aprobado, NO debe modificarse el orden, las etiquetas ni la estructura, solo codificarse):

```
+-----+
|               Centro Veterinario PatitasSanas               |
+-----+
| Registro de nuevo paciente                                   |
|                                                             |
| Nombre de la mascota:      [ _____ ]                  |
| Especie:                   [ _____ ]                  |
| Edad:                      [ _____ ]                  |
| Nombre del propietario:    [ _____ ]                  |
| Teléfono del propietario:  [ _____ ]                  |
|                                                             |
|               [ Registrar paciente ]                       |
+-----+
| Pacientes registrados                                       |
+-----+-----+-----+-----+-----+-----+-----+
| ID | Nombre mascota | Especie | Edad | Propietario | Tel. | |
Acción |
+-----+-----+-----+-----+-----+-----+-----+
| 1 | Firulais       | Perro  | 3    | Juan Pérez  | 3001..|
[Eliminar] |
+-----+-----+-----+-----+-----+-----+-----+
+-----+
```



El aprendiz debe respetar en su HTML:

- **Orden de los campos del formulario**, exactamente como se muestra: nombre de la mascota, especie, edad, nombre del propietario, teléfono del propietario.
- **Nombres de los campos (atributo `name` del input)**, coincidentes con las columnas de la tabla `pacientes`: `nombre_mascota`, `especie`, `edad`, `nombre_propietario`, `telefono_propietario`.
- **Texto exacto del botón** de registro: "Registrar paciente".
- **Texto exacto del botón** de eliminación en cada fila: "Eliminar".
- **Orden de las columnas** en la tabla de listado: ID, Nombre mascota, Especie, Edad, Propietario, Teléfono, Acción.

La hoja de estilos CSS (paleta de colores, tipografía, espaciados, distribución) sí queda a criterio del aprendiz, ya que el mockup entregado define la **estructura y el contenido**, no la apariencia visual final; esa libertad de estilo es intencional para que el aprendiz también aplique lo trabajado sobre CSS sin copiar un diseño exacto. No se exige el uso de un framework de interfaz en esta evaluación, dado el tiempo disponible.

INSTRUCCIONES DE ENTREGA

1. Cree en su computador la carpeta `evaluacion_patitassanas`.
2. Dentro de VS Code, cree los archivos: `app.py`, `database.py`, `models.py`, la carpeta `templates/` con `index.html`, y la carpeta `static/css/` con `estilos.css`.
3. Utilice **Flask** como framework para las rutas y **SQLite** como motor de base de datos.
4. Al finalizar, guarde todos los archivos y **no cierre el proyecto**; el instructor lo revisará directamente en su equipo.
5. Tiempo máximo: **2 horas**. Se recomienda distribuir así: 20 min. base de datos, 40 min. lógica CRUD y rutas, 40 min. interfaz HTML/CSS, 20 min. pruebas y ajustes finales.

REQUERIMIENTOS QUE DEBE CODIFICAR (siguiendo el diseño dado)

R1. Crear el archivo `database.py` que genere la base de datos `patitassanas.db` con la tabla `pacientes` descrita arriba, aplicando el tipo de dato correcto para cada atributo.

R2. Codificar en `models.py` las funciones necesarias para realizar el **CRUD completo** sobre la tabla `pacientes`:

- Registrar un nuevo paciente.
- Listar todos los pacientes.



- Eliminar un paciente por su `id`.
- (Opcional, si el tiempo lo permite: actualizar los datos de un paciente existente).

R3. Codificar en `app.py`, usando Flask, las rutas necesarias para:

- Mostrar el formulario y la lista de pacientes (`GET /`).
- Procesar el registro de un nuevo paciente (`POST`).
- Eliminar un paciente (`POST` a una ruta con el `id` del paciente).

R4. Aplicar **al menos dos medidas de seguridad en la codificación**, explicando en un comentario dentro del código cuál principio de seguridad se está aplicando en cada caso (por ejemplo: uso de consultas parametrizadas para evitar inyección SQL, validación de que la edad no sea negativa antes de insertar el registro, etc.).

R5. Construir la interfaz en `templates/index.html`, con el formulario y la tabla de pacientes ya registrados, enlazada a una hoja de estilos propia `static/css/estilos.css` (no se permite usar solo estilos en línea; debe existir el archivo CSS separado).

R6. El diseño visual en CSS debe aplicar como mínimo: una paleta de colores definida, estilo diferenciado para el formulario y para la tabla, y que la tabla sea legible (bordes o alternancia de color en las filas).

CRITERIOS DE ACEPTACIÓN (lo que el instructor verificará en su equipo)

- El formulario y la tabla de listado respetan el orden de campos, los nombres (`name`) y los textos de botón indicados en el mockup entregado (fidelidad al diseño ya aprobado, no se evalúa si el aprendiz "inventó" un diseño distinto).
 - La base de datos se crea correctamente al ejecutar el proyecto y contiene la tabla `pacientes` con los atributos y tipos definidos.
 - Es posible registrar un nuevo paciente desde el formulario y este se refleja de inmediato en la tabla/lista.
 - Es posible eliminar un paciente desde la interfaz.
 - Las consultas a la base de datos usan parámetros (no concatenación de cadenas).
 - El código incluye al menos una validación de datos antes de insertar en la base de datos.
 - El archivo `estilos.css` existe, está enlazado correctamente y aplica un diseño propio y coherente.
 - El código está mínimamente comentado, explicando las decisiones de seguridad tomadas.
-



PREGUNTAS DE ANÁLISIS CERRADAS SOBRE EL EJERCICIO REALIZADO

(Responda únicamente sobre el ejercicio que usted mismo acaba de codificar. Marque con una X. Vale 1 punto cada una — 10 puntos)

36. En la tabla `pacientes` que usted codificó, el atributo `id` cumple la función de:

- a) Clave foránea
- ☒ b) Clave primaria
- c) Índice de texto
- d) Variable de entorno

37. La instrucción SQL que usted utilizó para crear la tabla `pacientes` corresponde al tipo:

- a) DML (Data Manipulation Language)
- ☒ b) DDL (Data Definition Language)
- c) DCL (Data Control Language)
- d) TCL (Transaction Control Language)

38. Para evitar inyección SQL en la función que registra un nuevo paciente, usted debió:

- a) Concatenar directamente los valores del formulario dentro de la cadena SQL
- ☒ b) Usar marcadores de posición (parámetros) en la consulta, en lugar de concatenar texto
- c) Eliminar la validación de datos para agilizar el registro
- d) Guardar la consulta SQL en un archivo `.env`

39. La operación de su CRUD que corresponde a "eliminar un paciente" se identifica con la letra:

- a) C
- b) R
- c) U
- ☒ d) D

40. El archivo `estilos.css` que usted enlazó en `index.html` corresponde a:

- a) Lógica del servidor
- b) Estructura de la base de datos
- ☒ c) Presentación visual de la interfaz
- d) Definición de rutas de la aplicación

41. Si usted validó que el campo `edad` no puede ser negativo antes de insertar el registro, esa acción corresponde principalmente al principio de:



- a) Control de versiones
- ☒ b) Validación de entradas (seguridad en la codificación)
- c) Normalización de la base de datos
- d) Autenticación multifactor

42. La carpeta `templates/` en un proyecto Flask se utiliza para:

- a) Guardar los archivos de configuración del entorno virtual
- ☒ b) Guardar las páginas HTML que el servidor renderiza
- c) Guardar la base de datos SQLite
- d) Guardar las hojas de estilo CSS

43. Si al ejecutar `app.py` por segunda vez la tabla `pacientes` no se duplica ni genera error, esto se debe a que en la sentencia de creación se utilizó:

- a) `DROP TABLE`
- ☒ b) `CREATE TABLE IF NOT EXISTS`
- c) `DELETE FROM`
- d) `ALTER TABLE`

44. El método HTTP que usted debió usar para el formulario que registra un nuevo paciente es:

- a) GET
- ☒ b) POST
- c) DELETE
- d) PUT

45. ¿Cuál de las siguientes NO es una buena práctica que se le pidió aplicar en este ejercicio?

- a) Comentar el código explicando decisiones de seguridad
- b) Validar los datos antes de insertarlos en la base de datos
- ☒ c) Escribir consultas SQL concatenando directamente el texto ingresado por el usuario
- d) Usar consultas parametrizadas

Rúbrica de valoración de la sección práctica (criterios de aceptación)

Criterio de aceptación	Puntaje sugerido
Fidelidad al diseño entregado: orden de campos, <code>name</code> de los inputs y textos de botón exactamente como en el mockup	10



SERVICIO NACIONAL DE APRENDIZAJE SENA
Centro de Desarrollo Agroempresarial y Turístico del HUILA
Instrumento de evaluación conocimientos de proceso codificación
Tecnólogo en Análisis y Desarrollo de Software



Criterio de aceptación	Puntaje sugerido
Base de datos y tabla <code>pacientes</code> creadas correctamente con tipos y restricciones adecuadas	15
CRUD funcional (registrar, listar, eliminar como mínimo)	20
Consultas parametrizadas (sin concatenación insegura)	15
Al menos una validación de datos aplicada y comentada	10
Interfaz HTML funcional, enlazada correctamente al CSS propio	10
Calidad y coherencia del diseño CSS (paleta, formulario, tabla legible)	10
Comentarios que evidencian comprensión de las decisiones de seguridad tomadas	10
Total, sección práctica (criterios de aceptación)	100

JUICIO EVALUATIVO

Aprobado ☒

No Aprobado ☐

Firma instructor _____